

LDL08H01A

2008 年 8 月 1 日

第 1 章

論理譜

```

{ ===== }
{ 並び替え }
{ ===== }
logicname LDL08H01A

{ ===== }
{ 実効譜 }
{ ===== }
entity main
{ ----- }
{ 入力 }
{ ----- }
input reset;
input cs;
input re;
input we;
input exec;
input address[4];
inout data[8];
{ ----- }
{ 出力 }
{ ----- }
output eos;
{ ----- }
{ 内部信号 }
{ ----- }

bitr memory0d[8]; { メモリ 0 }
bitr memory1d[8]; { メモリ 1 }
bitr memory2d[8]; { メモリ 2 }
bitr memory3d[8]; { メモリ 3 }
bitr memory4d[8]; { メモリ 4 }
bitr memory5d[8]; { メモリ 5 }
bitr memory6d[8]; { メモリ 6 }
bitr memory7d[8]; { メモリ 7 }
bitr memory8d[8]; { メモリ 8 }
bitr memory9d[8]; { メモリ 9 }

bitn datain[8]; { 双方向データ入力 }
bitn dataout[8]; { 双方向データ出力 }
bitn datacnt; { 双方向データ制御 }

bitr state[3]; { 並び替え行程 }
bitr point[4]; { メモリ位置計数 }
bitr chg; { 交換指標 }
bitn swap01p; { メモリ 01 交換指標 }
bitn swap12p; { メモリ 12 交換指標 }

```

```

bitn  swap23p;      { メモリ 23 交換指標 }
bitn  swap34p;      { メモリ 34 交換指標 }
bitn  swap45p;      { メモリ 45 交換指標 }
bitn  swap56p;      { メモリ 56 交換指標 }
bitn  swap67p;      { メモリ 67 交換指標 }
bitn  swap78p;      { メモリ 78 交換指標 }
bitn  swap89p;      { メモリ 89 交換指標 }
bitn  state0p;      { 並び替え行程指標比較 }
bitn  state1p;      { 並び替え行程指標交換 }
bitn  stateend;     { 並び替え行程指標終了 }
bitr  hit[2];       { 交換記憶 }

{ ----- }
{  検査端子  }
{ ----- }

output T0P[8]; T0P=memory0d;
output T1P[8]; T1P=memory1d;
output T2P[8]; T2P=memory2d;
output T3P[8]; T3P=memory3d;
output T4P[8]; T4P=memory4d;
output T5P[8]; T5P=memory5d;
output T6P[8]; T6P=memory6d;
output T7P[8]; T7P=memory7d;
output T8P[8]; T8P=memory8d;
output T9P[8]; T9P=memory9d;
output T10P[3]; T10P=state;
output T11P[4]; T11P=point;
output T12P; T12P=chg;
output T13P; T13P=swap01p;
output T14P; T14P=swap12p;
output T15P; T15P=swap23p;
output T16P; T16P=swap34p;
output T17P; T17P=swap45p;
output T18P; T18P=swap56p;
output T19P; T19P=swap67p;
output T20P; T20P=swap78p;
output T21P; T21P=swap89p;
output T22P[8]; T22P=dataout;
output T23P[2]; T23P=hit;

{ ----- }
{  双方向  }
{ ----- }

enable(data,datain,dataout,datacnt)

datacnt=re&cs;

{ ----- }
{  出力代入  }
{ ----- }

eos=hit.1;

{ ----- }
{  メモリ 0  }
{ ----- }

if (reset)
memory0d=0;
else
if (cs)
if (address==0)
if (we)
memory0d=datain;
else
memory0d=memory0d;

```

```

        endif
    else
        memory0d=memory0d;
    endif
else
    if (swap01p)
        memory0d=memory1d;
    else
        memory0d=memory0d;
    endif
endif
endif
endif
{ ----- }
{   メモリ 1   }
{ ----- }
if (reset)
    memory1d=0;
else
    if (cs)
        if (address==1)
            if (we)
                memory1d=datain;
            else
                memory1d=memory1d;
            endif
        else
            memory1d=memory1d;
        endif
    else
        if (swap01p)
            memory1d=memory0d;
        else
            if (swap12p)
                memory1d=memory2d;
            else
                memory1d=memory1d;
            endif
        endif
    endif
endif
endif
{ ----- }
{   メモリ 2   }
{ ----- }
if (reset)
    memory2d=0;
else
    if (cs)
        if (address==2)
            if (we)
                memory2d=datain;
            else
                memory2d=memory2d;
            endif
        else
            memory2d=memory2d;
        endif
    else
        if (swap12p)
            memory2d=memory1d;
        else
            if (swap23p)
                memory2d=memory3d;
            else
                memory2d=memory2d;
            endif
        endif
    endif
endif

```

```

        endif
    endif
endif
endif

{ ----- }
{ メモリ 3 }
{ ----- }

if (reset)
    memory3d=0;
else
    if (cs)
        if (address==3)
            if (we)
                memory3d=datain;
            else
                memory3d=memory3d;
            endif
        else
            memory3d=memory3d;
        endif
    else
        if (swap23p)
            memory3d=memory2d;
        else
            if (swap34p)
                memory3d=memory4d;
            else
                memory3d=memory3d;
            endif
        endif
    endif
endif
endif

{ ----- }
{ メモリ 4 }
{ ----- }

if (reset)
    memory4d=0;
else
    if (cs)
        if (address==4)
            if (we)
                memory4d=datain;
            else
                memory4d=memory4d;
            endif
        else
            memory4d=memory4d;
        endif
    else
        if (swap34p)
            memory4d=memory3d;
        else
            if (swap45p)
                memory4d=memory5d;
            else
                memory4d=memory4d;
            endif
        endif
    endif
endif
endif

{ ----- }
{ メモリ 5 }
{ ----- }

```

```

{ ----- }
  if (reset)
    memory5d=0;
  else
    if (cs)
      if (address==5)
        if (we)
          memory5d=datain;
        else
          memory5d=memory5d;
        endif
      else
        memory5d=memory5d;
      endif
    else
      if (swap45p)
        memory5d=memory4d;
      else
        if (swap56p)
          memory5d=memory6d;
        else
          memory5d=memory5d;
        endif
      endif
    endif
  endif
}

{ ----- }
{ メモリ 6 }
{ ----- }

  if (reset)
    memory6d=0;
  else
    if (cs)
      if (address==6)
        if (we)
          memory6d=datain;
        else
          memory6d=memory6d;
        endif
      else
        memory6d=memory6d;
      endif
    else
      if (swap56p)
        memory6d=memory5d;
      else
        if (swap67p)
          memory6d=memory7d;
        else
          memory6d=memory6d;
        endif
      endif
    endif
  endif
}

{ ----- }
{ メモリ 7 }
{ ----- }

  if (reset)
    memory7d=0;
  else
    if (cs)
      if (address==7)
        if (we)
          memory7d=datain;

```

```

        else
            memory7d=memory7d;
        endif
    else
        memory7d=memory7d;
    endif
else
    if (swap67p)
        memory7d=memory6d;
    else
        if (swap78p)
            memory7d=memory8d;
        else
            memory7d=memory7d;
        endif
    endif
endif
endif
endif
{ ----- }
{ メモリ 8 }
{ ----- }
if (reset)
    memory8d=0;
else
    if (cs)
        if (address==8)
            if (we)
                memory8d=datain;
            else
                memory8d=memory8d;
            endif
        else
            memory8d=memory8d;
        endif
    else
        if (swap78p)
            memory8d=memory7d;
        else
            if (swap89p)
                memory8d=memory9d;
            else
                memory8d=memory8d;
            endif
        endif
    endif
endif
endif
{ ----- }
{ メモリ 9 }
{ ----- }
if (reset)
    memory9d=0;
else
    if (cs)
        if (address==9)
            if (we)
                memory9d=datain;
            else
                memory9d=memory9d;
            endif
        else
            memory9d=memory9d;
        endif
    else
        if (swap89p)

```



```

        memory9d=memory8d;
    else
        memory9d=memory9d;
    endif
endif
endif

{ ----- }
{ メモリ外部出力 }
{ ----- }

switch(address)
    case 0: dataout=memory0d;
    case 1: dataout=memory1d;
    case 2: dataout=memory2d;
    case 3: dataout=memory3d;
    case 4: dataout=memory4d;
    case 5: dataout=memory5d;
    case 6: dataout=memory6d;
    case 7: dataout=memory7d;
    case 8: dataout=memory8d;
    case 9: dataout=memory9d;
endswitch

{ ----- }
{ 並び替え行程 }
{ ----- }

if (reset)
    state=0;
else
    if (exec)
        if (stateend)
            state=0;
        else
            state=state+1;
        endif
    else
        state=0;
    endif
endif

{ ----- }
{ メモリ位置計数 }
{ ----- }

if (reset)
    point=0;
else
    if (stateend)
        if (point==8)
            point=0;
        else
            point=point+1;
        endif
    else
        point=point;
    endif
endif

{ ----- }
{ 並び替え行程指標 }
{ ----- }

if (exec)
    switch(state)
        case 0: state0p=1;
        case 1: state1p=1;
        stateend=1;
    endswitch
endif

```

```

        endswitch
    endif

{ ----- }
{ 交換指標 }
{ ----- }

    switch(point)
    case 0:
        if (state0p)
            if (memory0d>memory1d) chg=1; endif
        else
            chg=chg;
        endif
    case 1:
        if (state0p)
            if (memory1d>memory2d) chg=1; endif
        else
            chg=chg;
        endif
    case 2:
        if (state0p)
            if (memory2d>memory3d) chg=1; endif
        else
            chg=chg;
        endif
    case 3:
        if (state0p)
            if (memory3d>memory4d) chg=1; endif
        else
            chg=chg;
        endif
    case 4:
        if (state0p)
            if (memory4d>memory5d) chg=1; endif
        else
            chg=chg;
        endif
    case 5:
        if (state0p)
            if (memory5d>memory6d) chg=1; endif
        else
            chg=chg;
        endif
    case 6:
        if (state0p)
            if (memory6d>memory7d) chg=1; endif
        else
            chg=chg;
        endif
    case 7:
        if (state0p)
            if (memory7d>memory8d) chg=1; endif
        else
            chg=chg;
        endif
    case 8:
        if (state0p)
            if (memory8d>memory9d) chg=1; endif
        else
            chg=chg;
        endif
    endif
endswitch

{ ----- }
{ メモリ交換指標 }
{ ----- }

```

```

if (state1p)
    switch(point)
        case 0: if (chg) swap01p=1; endif
        case 1: if (chg) swap12p=1; endif
        case 2: if (chg) swap23p=1; endif
        case 3: if (chg) swap34p=1; endif
        case 4: if (chg) swap45p=1; endif
        case 5: if (chg) swap56p=1; endif
        case 6: if (chg) swap67p=1; endif
        case 7: if (chg) swap78p=1; endif
        case 8: if (chg) swap89p=1; endif
    endswitch
endif

{ ----- }
{   交換記憶   }
{ ----- }

if (reset)
    hit=0;
else
    if (exec)
        if ((state==1)&(point==8))
            switch(hit)
                case 0: hit=2;
                case 1: hit=0;
                case 2: hit=hit;
            endswitch
        else
            if (chg)
                hit=1;
            else
                hit=hit;
            endif
        endif
    else
        hit=0;
    endif
endif
ende

```

```

{ ===== }
{ 機能実行譜 }
{ ===== }
entity sim
{ ----- }
{ 検査端子 }
{ ----- }
output reset;
output cs;
output re;
output we;
output exec;
output address[4];
output data[8];
output eos;
{ ----- }
{ 内部信号 }
{ ----- }
bitr tc[10];

{ ----- }
{ 実効譜導入 }
{ ----- }
part main(reset,cs,re,we,exec,address,data,eos)

{ ----- }
{ 検査位置 }
{ ----- }
tc=tc+1;

{ ----- }
{ 初期化 }
{ ----- }
if (tc<5) reset=1; endif

{ ----- }
{ メモリ書き込み }
{ ----- }
switch(tc)
case 7: cs=1; we=1; data=10; address=0;
case 8: cs=1; we=1; data=9 ; address=1;
case 9: cs=1; we=1; data=8 ; address=2;
case 10: cs=1; we=1; data=7 ; address=3;
case 11: cs=1; we=1; data=6 ; address=4;
case 12: cs=1; we=1; data=5 ; address=5;
case 13: cs=1; we=1; data=4 ; address=6;
case 14: cs=1; we=1; data=3 ; address=7;
case 15: cs=1; we=1; data=2 ; address=8;
case 16: cs=1; we=1; data=1 ; address=9;
endswitch

{ ----- }
{ 並び替え実行指標 }
{ ----- }
if (tc>20) exec=1; endif

{ ----- }
{ メモリ読み出し }
{ ----- }
switch(tc)
case 205: cs=1; re=1; address=0;
case 206: cs=1; re=1; address=1;
case 207: cs=1; re=1; address=2;

```

```
        case 208: cs=1; re=1; address=3;
        case 209: cs=1; re=1; address=4;
        case 210: cs=1; re=1; address=5;
        case 211: cs=1; re=1; address=6;
        case 212: cs=1; re=1; address=7;
        case 213: cs=1; re=1; address=8;
        case 214: cs=1; re=1; address=9;
    endswitch
ende
endlogic
```